

贪吃蛇项目协作使用教程

面向小组成员：如何拉取仓库、认领模块、编译运行、提交代码和处理常见问题

项目	内容
仓库地址	https://github.com/zhxymj/tanchishe
实现语言	C 语言，源码统一为 .c 和 .h 文件
推荐编译器	Windows + GCC
协作方式	每人负责一个模块，尽量只修改自己负责的文件

1. 项目目标

本项目是 C 语言控制台版贪吃蛇。当前仓库已经搭好整体框架，主要目的是让 5 名成员可以分别完成自己的模块，而不是所有人挤在一个 `main.c` 文件里修改。

- ☐ 项目整体用 C 语言实现，不使用 C++、Python 或其他语言替代核心功能。
- ☐ 每个成员都有独立编码任务，提交时能体现自己的代码贡献。
- ☐ 公共结构体和函数接口已经建立，成员之间通过这些接口协作。
- ☐ 当前版本是可编译框架，核心逻辑由各成员根据 TODO 继续完善。

2. 文件分工

成员	负责文件	主要任务
成员1	<code>src/main.c</code> <code>src/game.c</code> <code>src/game.h</code>	主程序、游戏状态、初始化、暂停、重开、退出和模块整合
成员2	<code>src/snake.c</code> <code>src/snake.h</code>	蛇身数据结构、方向控制、蛇头移动、蛇身跟随和增长
成员3	<code>src/food.c</code> <code>src/food.h</code>	食物随机生成、避免与蛇身重合、得分、等级和速度变化
成员4	<code>src/collision.c</code> <code>src/collision.h</code>	撞墙检测、撞自身检测、失败判断和胜利判断
成员5	<code>src/ui.c</code> <code>src/ui.h</code>	地图绘制、蛇和食物显示、分数显示、键盘输入和界面刷新

原则：每个人优先只改自己负责的文件。如果必须改 `src/types.h` 或 `src/config.h`，要先和组员说明，因为这些文件会影响所有模块。

3. 第一次使用前准备

3.1 安装 Git

Git 用来下载项目、提交代码和推送到 GitHub。安装后在 PowerShell 输入下面命令，能显示版本号就说明安装成功。

```
git --version
```

3.2 安装 GCC

GCC 用来编译 C 语言代码。安装完成后输入：

```
gcc --version
```

如果提示找不到 gcc，需要把 GCC 的 bin 目录加入系统 Path，或者使用已经配置好 GCC 的终端。

3.3 GitHub 网络和代理

如果在国内访问 GitHub 很慢，可以先打开 v2rayN。你的代理常见端口如下：HTTP 为 10809，SOCKS 为 10808。PowerShell 需要手动设置代理：

```
$env:HTTP_PROXY="http://127.0.0.1:10809"  
$env:HTTPS_PROXY="http://127.0.0.1:10809"
```

验证 GitHub 是否能访问：

```
curl.exe -I -x http://127.0.0.1:10809 https://github.com
```

如果返回 HTTP/1.1 200 OK 或 200 Connection established，说明代理基本可用。

4. 克隆项目

选择一个你想保存项目的目录，例如桌面或 D 盘项目目录，然后运行：

```
git clone https://github.com/zhxymj/tanchishe.git  
cd tanchishe
```

拉取后应能看到这些文件：README.md、TASKS.md、CONTRIBUTING.md、build.bat、src 文件夹。

5. 编译和运行

在 tanchishe 项目根目录运行：

```
gcc src\main.c src\game.c src\snake.c src\food.c src\collision.c src\ui.c -o snake.exe  
.\snake.exe
```

也可以直接运行 build.bat：

```
.\build.bat
```

当前是协作框架版本，能运行不代表游戏功能已经完整。每个模块里的 TODO 完成后，游戏才会逐步变完整。

6. 如何完成自己的模块

打开自己负责的 .c 文件，搜索 TODO (Member X)。例如成员2负责蛇身模块，可以在 src/snake.c 里看到：

```
TODO (Member 2): initialize the full starting snake body  
TODO (Member 2): handle direction update rules  
TODO (Member 2): complete snake movement and growth
```

实现时注意：

- ☐ 优先保持 .h 文件里的函数名不变。
- ☐ 不要把所有代码写进 main.c，要放到自己负责的模块中。
- ☐ 写完一个小功能就编译一次，避免最后一次性出现很多错误。
- ☐ 如果需要新增公共字段，先确认其他成员不会受影响。

7. 同步最新代码

同步最新代码的意思是：把 GitHub 仓库中别人已经提交的内容下载到自己电脑上。每次开始写代码前都应该先同步一次，避免自己基于旧版本继续写。

7.1 写代码前的标准同步步骤

```
cd tanchishe
git status
git pull origin main
```

推荐顺序是先进入项目目录，再用 `git status` 看当前是否有未提交修改，最后运行 `git pull origin main` 拉取最新代码。

如果 `git status` 显示 `working tree clean`，说明当前没有未提交修改，可以放心 `pull`。

7.2 如果本地还没写代码

这是最简单的情况，直接同步：

```
git pull origin main
```

看到 `Already up to date` 表示你本地已经是最新；看到文件更新列表表示 Git 已经把别人的提交合并到你本地。

7.3 如果本地已经改了代码但还没提交

先不要直接 `pull`。先看自己改了哪些文件：

```
git status
```

如果这些修改是你准备保留的，有两种处理方式。推荐方式是先提交自己的修改，再同步远程代码：

```
git add src\你的模块.c src\你的模块.h
git commit -m "完成自己的阶段性修改"
git pull origin main
```

如果你只是临时改了一半，还不想提交，可以先暂存修改，再同步，最后恢复修改：

```
git stash push -m "临时保存未完成修改"
git pull origin main
git stash pop
```

`stash pop` 后如果出现冲突，需要手动解决冲突再继续编译。新手更推荐“先 `commit`，再 `pull`”。

7.4 如果 pull 后出现冲突

冲突通常说明两个人改到了同一个文件的同一段代码。先运行：

```
git status
```

Git 会列出冲突文件。打开冲突文件，找到 <<<<<<、=====、>>>>>> 这些标记，和对应组员确认保留哪部分。处理完后：

```
git add 冲突文件路径
```

```
git commit -m "解决同步冲突"
```

8. 提交代码到 GitHub

提交代码分为三步：add 选择要提交的文件，commit 在本地生成一次提交记录，push 把提交上传到 GitHub。

8.1 第一次提交前配置姓名和邮箱

每台电脑第一次使用 Git 提交代码时，需要配置提交者信息。把姓名换成自己的 GitHub 用户名或真实姓名：

```
git config --global user.name "你的名字"
```

```
git config --global user.email "你的邮箱"
```

这个配置只影响提交记录显示，不会改变代码内容。

8.2 查看改动范围

```
git status
```

重点看红色或绿色文件名，确认只包含自己负责的模块。如果出现不相关文件，不要提交它。

想看某个文件具体改了些什么，可以运行：

```
git diff src\snake.c
```

8.3 编译检查

```
gcc src\main.c src\game.c src\snake.c src\food.c src\collision.c src\ui.c -o snake.exe
```

编译成功后再提交。snake.exe 是编译产物，不需要提交，仓库已经用 .gitignore 忽略它。

8.4 只 add 自己负责的文件

不要习惯性使用 `git add .`，因为它可能把别人的文件、临时文件或无关文件一起提交。推荐明确写文件名。

例如成员2只提交蛇身模块：

```
git add src\snake.c src\snake.h
```

例如成员3只提交食物和计分模块：

```
git add src\food.c src\food.h
```

8.5 commit: 写清楚这次完成了什么

提交信息要简短明确, 例如:

```
git commit -m "完成蛇身移动模块"  
git commit -m "完成食物生成和计分模块"  
git commit -m "修复撞墙检测逻辑"
```

如果 commit 后提示 nothing to commit, 说明你还没有 git add 文件, 或者文件没有实际修改。

8.6 push: 上传到 GitHub

```
git push origin main
```

第一次 push 可能会弹出 GitHub 登录窗口, 按提示登录自己的 GitHub 账号即可。

如果终端网络连不上 GitHub, 先设置代理再 push:

```
$env:HTTP_PROXY="http://127.0.0.1:10809"  
$env:HTTPS_PROXY="http://127.0.0.1:10809"  
git push origin main
```

8.7 push 后确认是否成功

push 成功后, 终端通常会显示 main -> main。然后打开 GitHub 仓库页面刷新, 确认自己的提交出现在 commits 记录中。

也可以在本地查看最近提交:

```
git log --oneline -5
```

8.8 完整示例: 成员2提交蛇身模块

```
cd tanchishe  
git pull origin main
```

```
# 修改 src\snake.c 和 src\snake.h 后先编译  
gcc src\main.c src\game.c src\snake.c src\food.c src\collision.c src\ui.c -o snake.exe
```

```
# 检查、提交、推送  
git status  
git add src\snake.c src\snake.h  
git commit -m "完成蛇身移动模块"  
git push origin main
```

8.9 完整示例: 成员5提交界面模块

```
cd tanchishe  
git pull origin main
```

```
# 修改 src\ui.c 和 src\ui.h 后先编译  
gcc src\main.c src\game.c src\snake.c src\food.c src\collision.c src\ui.c -o snake.exe  
git status
```

```
git add src\ui.c src\ui.h
git commit -m "完成界面绘制和键盘输入模块"
git push origin main
```

9. 每个成员的建议实现顺序

成员	建议顺序
成员1	先保证 <code>game_init</code> 、 <code>game_handle_key</code> 、 <code>game_update</code> 的调用顺序清楚，再配合其他模块整合。
成员2	先完成蛇初始化，再完成方向控制，最后完成蛇身移动和增长。
成员3	先完成食物随机生成，再处理食物不与蛇身重合，最后完成计分、等级和速度调整。
成员4	先完成撞墙检测，再完成撞自身检测，最后补充失败和胜利判断。
成员5	先画基础地图，再显示蛇和食物，最后完善分数、提示文字、暂停和结束界面。

10. 常见问题处理

10.1 git push 失败，提示网络超时

通常是 PowerShell 没有走代理。先执行：

```
$env:HTTP_PROXY="http://127.0.0.1:10809"
$env:HTTPS_PROXY="http://127.0.0.1:10809"
```

如果还是失败，检查 v2rayN 是否正在运行，并确认 HTTP 端口是否为 10809。

10.2 git push 失败，提示先 pull

说明别人已经先提交了代码，你本地落后于远程仓库。先运行：

```
git pull origin main
```

如果没有冲突，再重新 push。

10.3 出现代码冲突

如果 Git 提示 `conflict`，说明两个人改到了同一段代码。先不要乱删内容，打开冲突文件，找到类似下面的标记：

```
<<<<<< HEAD
你的代码
=====
别人提交的代码
>>>>>> 分支名
```

和对应成员确认保留哪部分，删掉这些冲突标记后重新编译、提交。

10.4 编译提示 undefined reference

通常是函数声明和实现不一致，或者编译命令漏了某个 .c 文件。请使用教程里的完整编译命令。

10.5 中文显示乱码

当前控制台代码使用了 SetConsoleOutputCP(65001)。如果仍然乱码，可以先把终端编码切到 UTF-8：

```
chcp 65001
```

11. 提交前检查清单

- ☐ 只修改了自己负责的模块文件。
- ☐ 运行 `git status` 看过改动范围。
- ☐ 运行 GCC 编译命令，没有编译错误。
- ☐ 没有提交 `snake.exe`、临时文件、截图草稿或无关文件。
- ☐ `commit message` 能说明这次完成了什么模块或功能。
- ☐ `push` 后在 GitHub 页面刷新，确认自己的提交已经出现。

12. 最简命令汇总

第一次下载项目

```
git clone https://github.com/zhxymj/tanchishe.git
cd tanchishe
```

每次写代码前同步最新代码

```
git status
git pull origin main
```

编译

```
gcc src\main.c src\game.c src\snake.c src\food.c src\collision.c src\ui.c -o snake.exe
```

查看自己改了哪些文件

```
git status
git diff src\你的模块.c
```

只提交自己的模块，以成员2为例，不要乱用 `git add .`

```
git add src\snake.c src\snake.h
git commit -m "完成蛇身移动模块"
git push origin main
```

如果本地改了一半但暂时不能提交，又需要同步远程

```
git stash push -m "临时保存未完成修改"
git pull origin main
git stash pop
```


结论：每名成员按自己的文件分工完成 TODO，提交前先编译，提交时只提交自己的模块文件。这样项目能保持清晰，也方便报告中说明每个人的代码贡献。